RESCIENCE C

Replication / Computer Science

[Re] Comparison-count benefits in learning-augmented priority queues

Sahil Siddiqui

Received
–

Published
–

DOI
–

Abstract

Learning-augmented priority queues can reduce expensive true-key comparisons, but replicating that result requires precise advice, tie and counting conventions. We partially replicate the sorting experiments of Benomar and Coester with an independently authored C++ implementation checked against their released Python code. Across 11,340 runs, the class-quality and decay responses agree with the original findings; median pointwise relative errors are below 0.16% on every curve at the two main-paper sizes. Comparison benefits remain conditional on advice quality, with the uninformative class endpoint exceeding the analytic heap reference. A separate extension identifies historical node-map ownership: last-occurrence cleanup reduces drained-queue traced memory from 35.18 to 0.024 MiB, with the ownership effect repeated on a second host. Native graph controls favor a direct heap on all seven tested workloads on both hosts. The artifact separates replicated comparison-count evidence from these additional memory and runtime measurements and provides reproducible input generation, raw counts and verification procedures.

1 Scope and original result

Benomar and Coester study priority queues augmented by pointer predictions, rank predictions, and dirty comparisons [1]. Their sorting experiments measure clean key comparisons, rather than elapsed time. We target the three learning-augmented curves in Figures 2 and 3 of Section 5 and Figures 6 and 7 of Appendix F.1. This is a partial replication: the Fibonacci-heap and Bai–Coester sorting implementations, the graph experiments, and the theoretical proofs are outside the replicated subset.

The comparison-count model is valuable when true comparisons are costly. Predictions and integer-index operations have a different cost in that model. The replication therefore preserves the original metric; it does not reinterpret a reduction in clean comparisons as a claim of faster wall-clock execution. Practical-cost findings from our earlier artifact study are identified separately as an extension.

We reconstruct the methods from the paper and consult the released implementation to resolve details of ties, prediction generation, sentinel comparisons, and search behavior [1, 2]. The new engine is separately authored C++; it is not a clean-room implementation. The artifact records these dependencies and provides a small differential correctness gate against the pinned Python source.

Algorithms with predictions combine useful advice with guarantees under error: caching is an early example [3], and Mitzenmacher and Vassilvitskii survey the broader framework [4]. Empirical priority-queue studies emphasize that implementation and

Copyright © S. Siddiqui, released under a Creative Commons Attribution 4.0 International license.
Correspondence should be addressed to Sahil Siddiqui (somamasiddiqui11@gmail.com)

workload choices affect practical costs [5, 6]. Learned-index benchmarks evaluate prediction machinery against conventional indexes [7]; later studies examine when learned indexes help and compare learned, compressed and traditional integer indexes [8, 9]. These works motivate separating the replicated clean-comparison model from the memory/runtime extension. We reproduce a specific published sorting result, without proposing a new asymptotic queue bound or transferring learned-index benchmark conclusions directly to priority queues.

2 Reconstruction and comparison accounting

All sorting inputs contain distinct true keys. Offline predictions determine a stable order of insertion, with the previously inserted node used as the next search hint. Online rank predictions accompany keys in a uniformly random insertion order. The dirty/clean model first searches using predicted-order comparisons and then corrects that location using true comparisons. Tied advice and negative predicted ranks are retained.

We count true-key comparisons in skip-list search and correction, matching the released convention that comparisons with the head sentinel are free. Dirty comparisons are recorded separately. Sorting advice, querying the prediction-rank index, pointer manipulation, and validating the output are outside the clean-comparison count. Node heights follow the geometric distribution with promotion probability one half. Validation checks ordering and the exact comparator behavior on shared small inputs and heights.

The binary-heap curve is the authors' analytic reference, calculated from their released expression [2]. It is not a measured binary-heap implementation. We report ratios to that reference and the parameter intervals where they cross one, without assuming that every prediction quality beats it.

3 Experimental protocol and published reference

The grid uses n = 1,000, 10,000 and 100,000 distinct keys and 21 parameter values per prediction setting. Let T be floor(n sqrt(n)). At grid index i from 0 to 20, the class parameter is floor(i n / 20) and the decay parameter is floor(i T / 20). Each algorithm and cell receives 30 independent replicates. The n = 1,000 curves provide the released aggregate anchor, while the larger sizes target the main paper's figures. Seeds, generated-input hashes, code hashes and completeness checks accompany the observations.

The artifact clamps the plotted class parameter zero to one class. Its random cutpoints can coincide with the zero endpoint, so even the largest plotted class parameter is not defined as perfectly accurate advice. Decay starts from true ranks and perturbs uniformly selected items by a fair signed unit step. We preserve these conventions rather than silently substituting a different prediction generator.

For efficient endpoint generation, the decay implementation samples the counts of the 2n signed item categories from a multinomial distribution, then applies each item's positive minus negative count to its true rank. This has the same mathematical endpoint law as literal independent perturbation steps because sorting observes only the final ranks. The floating-point sampler and random-number stream differ from the released generator; no bitwise same-seed agreement is claimed. A literal mode supports small validation cases.

The pinned repository contains mean and standard-deviation arrays for the three learned methods at n = 1,000, but no corresponding per-run observations. Published curves at the larger sizes are recovered from the final PDF's vector paths using calibrated axis ticks. The original paper states that each plotted point averages 30 independent runs; the effective number of runs underlying the released aggregate files cannot be verified from those files. Aggregate dispersion is not treated as a known standard error.

We report pointwise relative differences and descriptive whole-curve errors, retaining all parameter values and identifying uncertainty from the new replicates separately.
The assessment is claim-specific: class-quality response, decay response, ratios to the analytic heap reference, and differences among prediction models. No arbitrary numerical agreement threshold determines a single success score. A failure to match one curve is investigated against generator, comparator and extraction conventions before it is interpreted as a failure to replicate the published finding.
The primary engine is separately authored C++17. Before execution, 666 small shared-input cases matched the pinned Python sorting functions exactly in cumulative clean and dirty counts and in complete comparator-call trace digests. The optimized and AddressSanitizer/UndefinedBehaviorSanitizer builds both passed. Four malformed-input controls failed closed, and six checks matched the released analytic heap expression.
Within each new condition, the three methods share the final rank-advice array, the geometric-height sequence by insertion step, and the insertion permutation where applicable. Offline sorting instead stably orders the ordinal keys by advice. Repetitions and parameter cells use independent, domain-separated advice, order and height streams. This pairing preserves each method's marginal experiment while differing from the original unseeded, interleaved algorithm loops.
The condition grid, source hashes, claim-specific analysis rules and published references were privately frozen before primary execution. This is a documented computational freeze, not an externally registered preregistration. The run executed on the Apple M1 MacBook Air under CPython 3.13.3 and NumPy 2.2.4, using Apple Clang 15 with C++17, -O3 and -DNDEBUG. Each model record constructs fresh queue state and checks the entire ordinal output. No sorting wall-time claim is made.
Pointwise intervals use 5,000 deterministic percentile-bootstrap resamples of our thirty runs. Paired model ratios resample shared conditions together; endpoint contrasts resample independently seeded endpoint cells separately. Intervals are pointwise, without simultaneous coverage correction. Every planned cell and recorded failure is retained, and no parameter or seed was retuned after observing the primary counts.

4 Replication results

All 3,780 planned shared-input conditions and 11,340 model runs completed, with exact ordinal-output checks passing for every run. This yields 378 model/parameter means with 30 replicates each. The new records and all earlier graph timing records are kept separate.
The curves closely follow the published comparison-count behavior. Across the twelve curves at the two main-paper sizes, median pointwise absolute relative errors range from 0.028% to 0.151%; the largest individual relative error is 0.743%. For the six 1,000-key released-aggregate curves, the largest individual error is 4.650%. Table 1 reports every curve, including the smaller-size deviations.
Figures 1 and 2 overlay the independently generated means and the published references at the main-paper sizes. Solid lines with open markers show this replication; dashed lines show the published means. The narrow shaded bands are pointwise 95% bootstrap intervals from our 30 replicates. No original standard errors or distributional-equivalence claim are inferred from these overlays.
All eighteen endpoint contrasts have the expected direction: increasing the class parameter from its first to last value reduces the mean clean count, while increasing decay from zero to its maximum increases it, for all three methods and all three sizes. Each pointwise 95% bootstrap interval for the endpoint difference excludes zero in that direction. This assesses endpoint responses; it does not require every adjacent noisy mean to be monotonic.
The advantage over the analytic heap reference depends on advice quality. Pointwise bootstrap intervals lie below that reference in 369 of 378 cells. The nine remaining cells

Table 1. Agreement of replicated means with the available references. At n = 1,000, reference values are released aggregate means; at larger sizes they are vector-derived plotted means from the final paper. Relative errors use the reference mean as denominator. Normalized RMSE is whole-curve RMSE divided by the arithmetic mean of its 21 reference means. These are descriptive agreement measures.

Setting	n	Model	Median rel. error (%)	Max rel. error (%)	Normalized RMSE (%)
Class	1,000	Offline	0.590	4.650	2.986
Class	1,000	Online	0.813	3.517	1.773
Class	1,000	Dirty/clean	0.681	2.030	1.116
Class	10,000	Offline	0.151	0.743	0.700
Class	10,000	Online	0.113	0.621	0.503
Class	10,000	Dirty/clean	0.101	0.596	0.738
Class	100,000	Offline	0.059	0.307	0.605
Class	100,000	Online	0.064	0.309	0.449
Class	100,000	Dirty/clean	0.028	0.207	0.296
Decay	1,000	Offline	0.646	2.189	1.004
Decay	1,000	Online	0.609	2.122	1.015
Decay	1,000	Dirty/clean	0.335	2.806	1.046
Decay	10,000	Offline	0.094	0.335	0.164
Decay	10,000	Online	0.067	0.450	0.195
Decay	10,000	Dirty/clean	0.087	0.259	0.135
Decay	100,000	Offline	0.032	0.124	0.055
Decay	100,000	Online	0.039	0.089	0.049
Decay	100,000	Dirty/clean	0.033	0.092	0.048

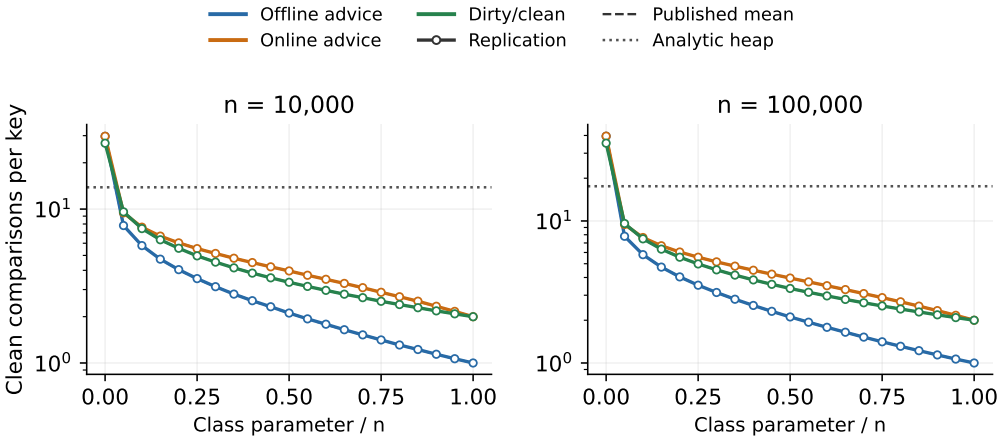


Figure 1. Partial replication of the original Figure 2: clean comparisons per key under class predictions. Both ordinate axes are logarithmic. The plotted class parameter zero is implemented as one class. Published and replicated curves largely overlap at this scale. The dotted horizontal line is the released analytic binary-heap reference, not a timed heap.

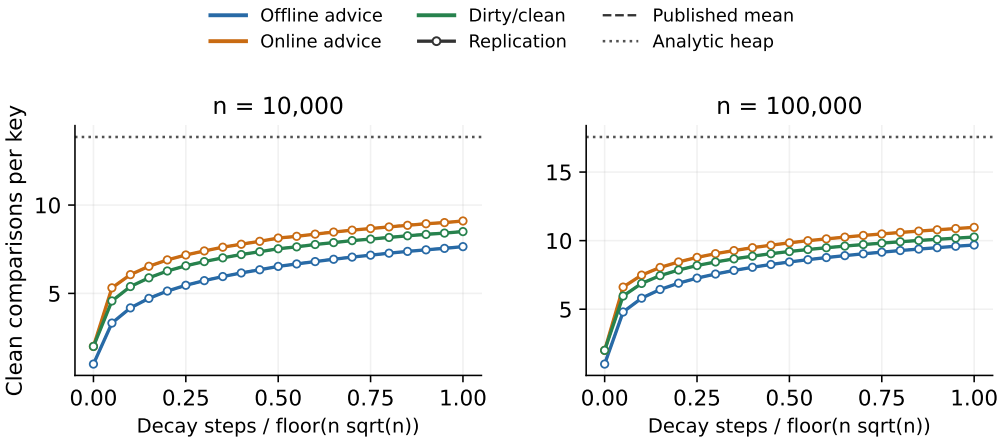


Figure 2. Partial replication of the original Figure 3: clean comparisons per key under decay predictions. The horizontal coordinate divides the timestep parameter by floor(n sqrt(n)); this rescales the original main-paper axis without changing the experiments. Better advice is at the left. Published and replicated curves largely overlap at this scale. The dotted line is the analytic binary-heap reference.

Table 2. Replicated mean clean comparisons per key at the first and last parameter values. Class start/end correspond to plotted c = 0/n; decay start/end correspond to t = 0/floor(n sqrt(n)). The full pointwise and endpoint interval tables are in the artifact.

n	Model	Class start	Class end	Decay start	Decay end	Analytic BH
1,000	Offline	19.740	1.000	0.999	5.736	9.954
1,000	Online	19.948	1.989	1.985	7.321	9.954
1,000	Dirty/clean	18.516	1.987	1.985	6.731	9.954
10,000	Offline	29.698	1.000	1.000	7.647	13.854
10,000	Online	29.738	1.998	1.998	9.099	13.854
10,000	Dirty/clean	26.801	1.998	1.998	8.498	13.854
100,000	Offline	39.497	1.000	1.000	9.681	17.557
100,000	Online	39.573	2.000	2.000	10.968	17.557
100,000	Dirty/clean	35.305	2.000	2.000	10.262	17.557

are the lowest-quality class endpoint, across all three sizes and methods; their intervals lie above it. All twenty other sampled class levels and all sampled decay levels lie below the reference. Thus the replication supports substantial comparison-count benefits over the tested useful-advice range while preserving the unfavorable uninformative-class endpoint visible in the original curves.

The relative ordering of prediction interfaces is also preserved: at the highest class parameter and at zero decay, offline advice requires about one clean comparison per key, and online and dirty/clean advice require about two. Model-to-model paired ratios and every observed heap-crossing bracket are reported in the artifact. The original qualitative description of comparable performance is not converted into an arbitrary numerical equivalence margin.

Original per-run observations and random states are not provided in the pinned artifact. The paper states a nominal thirty runs per plotted point, while the saved 1,000-key aggregates do not document their effective trial count. The published n = 100,000 class heap line corresponds to approximately 17.557000 comparisons per key; the released analytic expression gives 17.557310. We retain that small plot/reference discrepancy and calibrate the vector-derived means using the printed axis ticks, rather than adjusting a scale to force agreement.

5 Separate extension: historical ownership and runtime

The accompanying prior study extends the released artifact to memory ownership and runtime. These observations are not replications of the original sorting comparison figures. The full extension manuscript, raw observations and host-specific analyses remain a separate runtime/memory supplement; its findings are summarized here to clarify the boundary between the replicated cost model and practical execution costs.

Along a successful execution with finite numeric priorities, let H be the number of distinct priorities inserted during the lifetime of an original queue. The released implementation assigns each newly encountered priority an entry in sl.nodes and never erases that entry during deletion. Starting from an empty map, induction on insertions and extractions gives H map entries after the queue drains. Consequently, this metadata has an $\Omega(H)$ entry-count lower bound even when the maximum number of simultaneously live queue entries is fixed.

We test the mechanism independently of graph size with unique increasing priorities, constant rank prediction zero, index universe one, and at most 128 simultaneously live entries. Each method and insertion count runs in a fresh process. Tracing starts after imports and class warmup. After all entries are extracted, full garbage collection occurs while the empty queue object remains alive. Statistics bookkeeping contributes a small amount to traced allocations. All 12 method/length runs return the exact expected extraction sequence.

The safe ownership change removes a key's node-map entry after its final live occurrence is extracted, while retaining node links that may still support hints. Blindly detaching those links is a different intervention. The runtime study therefore distinguishes index substitution, ownership cleanup and the combined duplicate-grouping repair; it does not attribute all of their effects to a single change [2, 10].

In the prior bounded-live test with 100,000 insertions and at most 128 simultaneously live entries, last-occurrence map cleanup reduces post-GC retained traced memory from 35.18 to 0.02386 MiB on the Mac and from 34.42 to 0.02375 MiB on the Windows host. These are drained-queue retention measurements, not peak training or graph-query memory.

On the Mac, the direct heap has the lowest graph-level median among all tested native methods on every target. Guided bitmap + linear costs 1.64 to 10.63 times the direct heap. Bitmap/set linear-advisor ratios range from 0.800 to 0.995, but the lopsided case's

exploratory interval includes parity. Guided bitmap + linear/plain grouped-skiplist ratios range from 1.008 to 1.185, with the New York interval also including parity. All six guided variants have slower graph-level point estimates than the plain grouped skip list. An improved predecessor index therefore does not demonstrate a benefit from advice itself in this implementation.

The direct heap has the lowest workload-level median on all seven native targets on both hosts under both CPU and wall clocks. Across the full extension, 12,300 graph-query timing records are retained. That total is separate from the 11,340 counted sorting records in the present replication; the sorting records are not runtime measurements.

The two-host native study also compares a direct heap interface with shared scalar and grouped interfaces. Those controls make the application's routing costs visible. Host results remain separate: compiler, operating system and processor all differ, so timing differences do not isolate an architecture effect. Detailed tables, uncertainty and all exceptions are retained in the extension supplement.

6 Limitations and conclusion

The replication covers a specified sorting subset and a clean-comparison metric. It does not test every original algorithm, dataset or theorem. Published aggregate records and PDF coordinates offer less information than original per-run data. The independent implementation resolves documented ambiguities with the released source, and therefore shares those declared conventions. The separate practical-cost extension has a limited workload/host sample and does not establish universal runtime ordering.

Within the declared sorting subset, the comparison-count benefits replicate closely across both prediction-error models, three queue interfaces and three input sizes. The full curves preserve both useful-advice gains and the unfavorable uninformative-class endpoint. The independent implementation, explicit counting convention and published-reference extraction make that assessment reproducible. The separate ownership and native-runtime studies extend the artifact evaluation under different workloads and advice interfaces; they do not replace the original comparison metric or invalidate its theoretical cost model. For maintainers, the ownership fix is to delete a key's sl.nodes entry when its last live occurrence is extracted; node links can be left intact. The replication and extension together make both the published comparison result and practical implementation costs reproducible.

Artifact availability

The accompanying artifact contains source, raw observations, input hashes, frozen computational plans, correctness checks, and reproduction commands. Upstream source and road data are fetched separately by pinned hashes. The feasibility pilot is excluded from reported counts. AI coding assistants were used as tools for code generation, debugging, and editing of prose. The author designed the experiments, ran all measurements on both hosts, verified all reported figures against the raw artifact, and is responsible for all claims.

References

1. Z. Benomar and C. Coester. "Learning-Augmented Priority Queues." In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024, pp. 124163–124197. URL: https://papers.neurips.cc/paper_files/paper/2024/file/e08e1a60c006ac3f0c9f953626b0f0c8-Paper-Conference.pdf.

2. Z. Benomar. *Learning-augmented-priority-queues*. Released source, pinned commit 3915cc261e87949b2b0aa724673c2227492d1b08. 2024. URL: https://github.com/Ziyad-Benomar/Learning-augmented-priority-queues/tree/3915cc261e87949b2b0aa724673c2227492d1b08.
3. T. Lykouris and S. Vassilvitskii. "Competitive Caching with Machine Learned Advice." In: *International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. 2018, pp. 3296–3305. URL: https://proceedings.mlr.press/v80/lykouris18a.html.
4. M. Mitzenmacher and S. Vassilvitskii. "Algorithms with Predictions." In: *Beyond the Worst-Case Analysis of Algorithms*. Ed. by T. Roughgarden. Cambridge University Press, 2021. Chap. 30, pp. 646–662. DOI: 10.1017/9781108637435.037. URL: https://arxiv.org/abs/2006.09123.
5. M. Chen, R. A. Chowdhury, V. Ramachandran, D. L. Roche, and L. Tong. *Priority Queues and Dijkstra's Algorithm*. Tech. rep. TR-07-54. University of Texas at Austin, 2007. URL: https://www3.cs.stonybrook.edu/~rezaul/papers/TR-07-54.html.
6. D. H. Larkin, S. Sen, and R. E. Tarjan. "A Back-to-Basics Empirical Study of Priority Queues." In: *Meeting on Algorithm Engineering and Experiments*. 2014, pp. 61–72. DOI: 10.1137/1.9781611973198.7. URL: https://arxiv.org/abs/1403.0252.
7. A. Kipf, R. Marcus, A. van Renen, M. Stoian, A. Kemper, T. Kraska, and T. Neumann. "SOSD: A Benchmark for Learned Indexes." In: *NeurIPS Workshop on Machine Learning for Systems*. 2019. URL: https://www.cl.cam.ac.uk/~ey204/teaching/ACS/R244_2021_2022/papers/kipf_NeurIPS_2019.pdf.
8. Q. Liu, S. Han, Y. Qi, J. Peng, J. Li, L. Lin, and L. Chen. "Why Are Learned Indexes So Effective but Sometimes Ineffective?" In: *Proceedings of the VLDB Endowment* 18 (2025). URL: https://www.vldb.org/pvldb/vol18/p2886-liu.pdf.
9. L. Bellomo, G. Cianci, L. de Rosa, P. Ferragina, and M. Odorisio. "A Comparative Study of Compressed, Learned, and Traditional Indexing Methods for Integer Data." In: *Symposium on Experimental Algorithms*. 2025. DOI: 10.4230/LIPIcs.SEA.2025.5. URL: https://drops.dagstuhl.de/storage/00lipics/lipics-vol338-sea2025/html/LIPIcs.SEA.2025.5/LIPIcs.SEA.2025.5.html.
10. S. Siddiqui. *OnlineSL retains historical sl.nodes entries after the final occurrence is extracted*. GitHub issue 1, opened 6 November 2026. 2026. URL: https://github.com/Ziyad-Benomar/Learning-augmented-priority-queues/issues/1.